

MINI PROJECT PPT

SUBMITTED BY: ALIZA AKBAR

ROLL NO.: 202410101150222

COURSE: B. TECH CS(DS+AI), II - YEAR

GROUP: CS - DS 47,48

SEMESTER: IV

SUBJECT: IT WORKSHOP LAB

SUBJECT CODE: UCS4801

SUBMITTED TO: ER. ABHISHEK KUMAR SAXENA

DR. AKHILESH KUMAR GUPTA





Hospital Patient Management System Using Scilab

A practical guide to implementing an automated patient management system for healthcare facilities using open-source numerical computing software.

Project Objectives



System Creation

Develop a functional Hospital Patient Management System using Scilab programming environment



Data Storage

Store comprehensive patient details including demographics and medical information



Diagnosis Tracking

Monitor patient diagnoses and treatment records systematically



Financial Management

Track consultation fees and calculate total revenue collected



Record Display

Generate formatted patient records and summary reports

System Requirements

Software Stack

- Scilab (open-source numerical computing platform)
- Compatible with Windows or Linux operating systems
- No additional software libraries required

Hardware Specifications

- Standard computer or laptop
- Minimum 2GB RAM recommended
- 100MB disk space for installation

Technical Skills Required

- Basic programming concepts (variables, loops, conditionals)
- Familiarity with Scilab syntax and command-line interface
- Understanding of matrix operations and data structures
- Input/output handling in Scilab

Theoretical Foundation

Patient Management System Overview

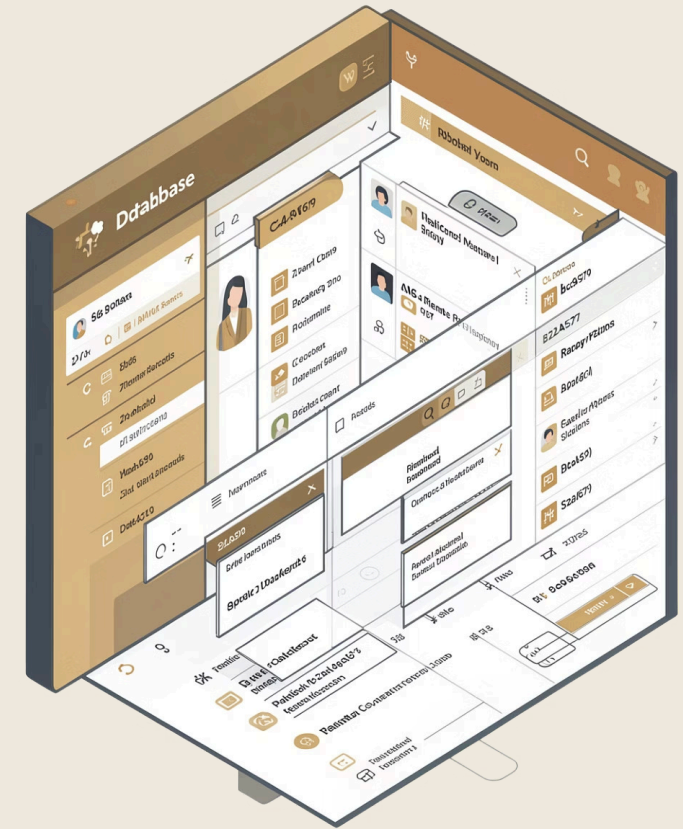
A centralized system that electronically stores, organizes, and manages comprehensive patient records, replacing error-prone manual documentation methods.

Data Organization Principles

Matrices efficiently store numerical data elements such as patient age and consultation fees, while structures handle text-based information like names and diagnoses.

User Interaction Design

A simple command-line interface enables healthcare staff to input new patient data and retrieve existing records through intuitive menu-driven operations.



Core Programming Concepts



Matrix Data Storage

Two-dimensional arrays store numerical patient data including age values and consultation fees, enabling efficient mathematical operations and data retrieval.



Structure Implementation

Custom data structures organize related patient information fields such as name, diagnosis, and contact details into coherent record formats.



Loop Statements

Iterative loops process multiple patient records sequentially, automating data entry and display operations for batch processing efficiency.



I/O Functions

Built-in Scilab functions handle user input collection and formatted output display, creating an interactive system interface.



Interface Design

Menu-driven command-line interface provides intuitive navigation for data entry, record retrieval, and summary generation operations.

Implementation Algorithm



Program Initialization

Clear workspace and initialize variables to ensure clean execution environment

1

Patient Count Input

Request user to specify number of patients for data entry processing



Details Collection

Loop through patient entries, gathering name, age, diagnosis, and fees for each



Data Storage

Store collected information in appropriate matrix and structure formats



Record Display

Print formatted patient records with all collected details for verification



Total Calculation

Sum all consultation fees to generate total revenue collected from patients



Program Completion

Display summary and terminate execution gracefully

System Flowchart



Program Code & Sample Output

Implementation Snippet

```
clc;
clear;

// Number of patients
n = input("Enter number of patients: ");

// Initialize arrays
name = emptystr(n,1);
age = zeros(n,1);
diagnosis = emptystr(n,1);
fees = zeros(n,1);

// Input patient details
for i = 1:n
    disp("Enter details for patient " + string(i));

    name(i) = input("Enter patient name: ", "string");
    age(i) = input("Enter age: ");
    diagnosis(i) = input("Enter diagnosis: ", "string");
    fees(i) = input("Enter fees: ");
end

// Display patient records
disp("Patient Records:");

for i = 1:n
    disp("Patient " + string(i));
    disp("Name: " + name(i));
    disp("Age: " + string(age(i)));
    disp("Diagnosis: " + diagnosis(i));
    disp("Fees: " + string(fees(i)));
end

// Generate summary
totalFees = sum(fees);

disp("Total Fees Collected:");
disp(totalFees);
```

Sample Output

Enter number of patients: 2

"Enter details for patient 1"

Enter patient name: Ayush

Enter age: 30

Enter diagnosis: Fever

Enter fees: 500

"Enter details for patient 2"

Enter patient name: Rohan

Enter age: 58

Enter diagnosis: Heart Checkup

Enter fees: 1200

"Patient Records:"

"Patient 1"

"Name: Ayush "

"Age: 30"

"Diagnosis: Fever"

"Fees: 500"

"Patient 2"

"Name: Rohan"

"Age: 58"

"Diagnosis: Heart Checkup"

"Fees: 1200"

"Total Fees Collected:" 1700.

System Benefits & Applications

Operational Advantages

- **Streamlined Record Management**
Centralized digital storage eliminates manual filing systems and paper-based documentation
- **Time Efficiency**
Automated data entry and retrieval reduces administrative processing time significantly
- **Reduced Paperwork**
Digital records minimize physical documentation requirements and storage space
- **Improved Accuracy**
Structured data entry reduces human transcription errors and data inconsistencies

Healthcare Applications

Hospital Wards Inpatient and outpatient department management	Clinic Operations Private practice patient tracking systems
Medical Centers Multipurpose healthcare facility coordination	Healthcare Networks Integrated system for multiple facility management

Conclusion & Key Takeaways



System Capabilities

Successfully stores comprehensive patient details including demographic information, medical diagnoses, and financial data while calculating total fees collected across all patients.

Real-World Application

Demonstrates practical healthcare management principles using accessible open-source Scilab software, bridging programming concepts with medical operations workflows.

Educational Value

Serves as foundational learning tool for developing basic programming skills, understanding data storage methodologies, and applying computational thinking to healthcare challenges.



Thank You

For Your Time and Attention